

LINUX LAB REPORT 38 'AI'

Prepared By:
Rupesh Yadav

Presented To:
Rupak Rajbanshi



Table Of Contents

I. File Operations.....	2
II. APT Package Management	5
III. C Compilation	6
A. Step 1: Write the Code.....	6
B. Step 2: Compile and Run	7
C. Step 3: Modify and Recompile	8
IV. Cleanup and Verification	9
Summary	10

Objective: This lab covers essential Linux file operations, package management using APT, and compiling/running a simple C program using GCC on a Kali Linux system. Each command below is explained in two lines and followed by a placeholder for a screenshot of the corresponding terminal output.

I. File Operations

```
mkdir ~/linux_lab
```

This command creates a new directory named "linux_lab" inside the home folder.

It is the first step to keep all lab files organized in one place.

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ dir
aayush.c  folders  hello.c  hello.c.save  output  output.c  rupesh.c
(kali@kali)-[~/Desktop]
$ mkdir linux_lab
```

```
cd ~/linux_lab
```

This command changes the current working directory to the newly created linux_lab folder.

All subsequent commands in this section are run from inside this directory.

```
(kali@kali)-[~/Desktop]
$ cd linux_laab
cd: no such file or directory: linux_laab
(kali@kali)-[~/Desktop]
$ cd linux_lab
```

```
nano greeting.txt (type: "Hello Linux!" then Ctrl+O, Enter, Ctrl+X)
```

This opens the nano text editor to create a new file called greeting.txt.

The text "Hello Linux!" is typed in, saved with Ctrl+O, and the editor is closed with Ctrl+X.

```
cat greeting.txt
```

This command displays the contents of greeting.txt directly in the terminal.

It confirms that the file was created and saved correctly.

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop/linux_lab]
$ nano greeting.txt
(kali@kali)-[~/Desktop/linux_lab]
$ cat greeting.txt
Hello linux!
```

```
cp greeting.txt backup.txt
```

The cp command copies greeting.txt into a new file called backup.txt.

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop/linux_lab]
$ cp greeting.txt backup.txt
```

`mv backup.txt old_greeting.txt`

The mv command then renames backup.txt to old_greeting.txt.

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop/linux_lab]
$ mv backup.txt old_greeting.txt

(kali@kali)-[~/Desktop/linux_lab]
$ dir
greeting.txt  old_greeting.txt
```

`rm old_greeting.txt` (removes directory and its contents)

rm deletes the old_greeting.txt file, while mkdir/rmdir create and remove an empty directory.

rm -r is used instead of rmdir when a directory still contains files, since rmdir only works on empty folders

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop/linux_lab]
$ rm old_greeting.txt

(kali@kali)-[~/Desktop/linux_lab]
$ dir
greeting.txt
```

`mkdir temp_dir`

The mkdir tempdir creates a new new empty directory named as temp_dir in current location.

`rmdir temp_dir`

rmdir remove the empty directory only . If there is anything inside directory. It will show error as “Directory not empty”

`# rm -r temp_dir`

Removes tempdir and everything inside it even if its not empty (the r flag means recursive)

```
(kali@kali)~[~/Desktop/linux_lab]
$ mkdir temp_dir

(kali@kali)~[~/Desktop/linux_lab]
$ rmdir temp_dir

(kali@kali)~[~/Desktop/linux_lab]
$ ls -all
total 12
drwxrwxr-x 2 kali kali 4096 Sep  9 03:04 .
drwxr-xr-x 4 kali kali 4096 Sep  9 03:02 ..
-rw-rw-r-- 1 kali kali 13 Sep  9 03:03 greeting.txt
```

```
ls -l greeting.txt
```

This command lists detailed information about greeting.txt, including permissions, owner, and size. It is used to verify file permissions before or after making changes.

```
(kali@kali)~[~/Desktop/linux_lab]
$ ls -l greeting.txt
-rw-rw-r-- 1 kali kali 13 Sep  9 03:03 greeting.txt
```

II. APT Package Management

Install the tools needed for the rest of the lab.

```
sudo apt update
```

This command refreshes the local list of available packages from the configured repositories. It should always be run first so that installations use the latest available package versions.

```
Session Actions Edit View Help

(kali@kali)~[~/Desktop/linux_lab]
$ sudo apt update
[sudo] password for kali:
Get:1 http://mirrors.ustc.edu.cn/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.ustc.edu.cn/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://mirrors.ustc.edu.cn/kali kali-rolling/main amd64 Contents (deb) [53.7 MB]
Get:4 http://mirrors.ustc.edu.cn/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:5 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:6 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.4 MB in 19s (4,095 kB/s)
1244 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
sudo apt install build-essential
```

This installs the build-essential package, which includes GCC, G++, make, and other compiling tools. It provides everything needed to compile C and C++ programs on the system.

```
Session Actions Edit View Help
(kali@kali)~[~/Desktop/linux_lab]
$ sudo apt install build-essential
build-essential is already the newest version (12.12).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1244
```

gcc --version

This command checks whether GCC was installed successfully and displays its version number. It is a quick way to confirm the compiler is ready for use.

```
Session Actions Edit View Help
(kali@kali)~[~/Desktop/linux_lab]
$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

apt search python3

This searches the package repository for any packages related to python3. It demonstrates how to look up and discover other available software packages.

```
Session Actions Edit View Help
(kali@kali)~[~/Desktop/linux_lab]
$ apt search python3

alembic/kali-rolling 1.18.4-2 all
  lightweight database migration tool for SQLAlchemy

antimony/kali-rolling 0.9.3-3+b1 amd64
  Computer-aided design CAD tool

astro-frameworks/kali-rolling 5.0 all
  Astronomical data analysis frameworks

astro-python3/kali-rolling 5.0 all
  Python 3 packages for astronomy

bamkit/kali-rolling 0.0.1+git20170413.ccd079d-3 all
  tools for common BAM file manipulations

beancount/kali-rolling 3.2.0-2 all
  Double-entry accounting from text files

berrynet-dashboard/kali-rolling 3.10.2-2 all
  deep learning gateway - python3 modules

brltty/kali-rolling 6.9.1+repack-1 amd64
  Access software for a blind person using a braille display
```

III. C Compilation

A. Step 1: Write the Code

nano hello.c

This opens nano to create the source file hello.c, which contains a simple C program.

The program declares two integers, prints a greeting, and prints their sum before returning 0.

```
(kali@kali)-[~/Desktop/linux_lab]
$ nano hello.c

GNU nano 9.2 hello.c
#include <stdio.h>
int main(){
    int a = 10, b = 5;
    printf("Hello, linux_lab!\n");
    printf("Sum of %d and %d is %d\n", a, b, a+b);
    return 0;
}
```

Session Actions Edit View Help

[Read 7 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

B. Step 2: Compile and Run

```
gcc hello.c -o hello_program
```

This compiles hello.c using GCC and names the output executable hello_program (-o specifies the name).

If there are no errors, an executable file is created in the current directory.

```
(kali@kali)-[~/Desktop/linux_lab]
$ gcc hello.c -o hello_program
```

```
ls -l hello_program
```

This lists the hello_program file to confirm that the executable was actually created.

The permissions column also shows whether the file is executable.

```
(kali@kali)-[~/Desktop/linux_lab]
$ ls -l hello_program
-rwxrwxr-x 1 kali kali 16000 Sep  9 03:44 hello_program
```

```
./hello_program
```

This runs the compiled program from the current directory.
It should print the greeting message and the sum of the two integers to the terminal.

```
(kali@kali)-[~/Desktop/linux_lab]
$ ./hello_program
Hello, linux_lab!
Sum of 10 and 5 is 15
```

```
gcc -Wall hello.c -o hello_program_warn
./hello_program_warn
```

The `-Wall` flag enables all common compiler warnings, which is considered best practice.
This helps catch potential bugs, such as unused variables or type mismatches, before they cause problems.

```
(kali@kali)-[~/Desktop/linux_lab]
$ nano hello.c
(kali@kali)-[~/Desktop/linux_lab]
$ gcc -Wall hello.c -o hello_program
(kali@kali)-[~/Desktop/linux_lab]
$ ./hello_program
Hello, linux_lab!
Sum of 200 and 5 is 205
```

C. Step 3: Modify and Recompile

```
nano hello.c (change a=100 to a=200, save and exit)
```

This reopens the source file so the value of variable 'a' can be changed from 100 to 200.
Editing and recompiling demonstrates the typical iterative development cycle.

```
(kali@kali)-[~/Desktop/linux_lab]
$ nano hello.c
```

```
Session Actions Edit View Help
GNU nano 9.2 hello.c *
#include <stdio.h>
int main(){
    int a = 200; b = 5;
    printf("Hello, Linux_lab!\n");
    printf("Sum of %d and %d is %d\n", a, b, a+b);
    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^I Execute
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify
```

```
gcc -Wall hello.c -o hello_program
./hello_program
```

The program is recompiled with the updated value using the same -Wall flag. Running it again confirms that the new sum reflects the updated value of 'a'.

```
(kali@kali)-[~/Desktop/linux_lab]
$ gcc -Wall hello.c -o hello_program

(kali@kali)-[~/Desktop/linux_lab]
$ ./hello_program
Hello, Linux_lab!
Sum of 200 and 5 is 205
```

IV. Cleanup and Verification

Ensure you understand what was created and clean up.

```
cd ~
Here i did cd .. to step back at the home directory.
```

This command returns to the home directory from wherever the user currently is. It is a common way to reset the working location before final checks.

```
(kali@kali)-[~/Desktop/linux_lab]
$ cd ..

(kali@kali)-[~/Desktop]
$
```

```
ls -lh ~/linux_lab/
```

This lists all files inside the linux_lab directory with human-readable file sizes (-h). It gives a full overview of everything created during the lab.

```
(kali@kali)-[~/Desktop]
$ ls -lh linux_lab
total 40K
-rw-rw-r-- 1 kali kali 13 Sep 9 03:03 greeting.txt
-rw-rw-r-- 1 kali kali 148 Sep 9 06:02 hello.c
-rwxrwxr-x 1 kali kali 16K Sep 9 06:03 hello_program
-rwxrwxr-x 1 kali kali 16K Sep 9 04:19 hello_program_warn
```

```
cat ~/linux_lab/hello.c
```

This displays the final content of hello.c one last time. It verifies that the source code changes were saved correctly.

```
(kali@kali)-[~/Desktop]
$ cat linux_lab/hello.c
#include <stdio.h>
int main(){
    int a = 200, b = 5;
    printf("Hello, Linux_lab!\n");
    printf("Sum of %d and %d is %d\n", a, b, a+b);
    return 0;
}
```

```
rm -rf ~/linux_lab (optional)
```

This command permanently deletes the entire linux_lab directory and everything inside it. It is optional and only used if a full cleanup of the lab environment is desired.

```
(kali@kali)-[~/Desktop]
$ rm -rf linux_lab

(kali@kali)-[~/Desktop]
$ dir
aayush.c folders hello.c hello.c.save output output.c rupesh.c

(kali@kali)-[~/Desktop]
$
```

Summary

Basic Linux file operations were performed in this lab by creating, viewing, copying, moving, renaming, and deleting files and directories through commands such as mkdir, cd, nano, cat, cp, mv, rm, and rmdir. The APT package manager was used to update the packages and install build-essential, which gave us the GCC compiler necessary for the second part of the lab.

We wrote and ran a basic C program (hello.c) with the help of GCC compiler. We used -Wall parameter to turn on warning messages in our compilation. Then, we made some modifications to our code and recompiled it, which is the usual cycle of editing, compiling, and running programs. In the end, we checked the directory to make sure all files were created properly and we showed an optional command to delete the lab directory.

All in all, we practiced some basic Linux commands in this lab, such as managing files and directories, installing packages via APT package manager, and compiling and running our C code with GCC..